Practical Python Questions for Cisco Automation Engineers

Number Data Type

1. Beginner: Write a Python script to calculate the total bandwidth usage given a list of interface speeds (in Mbps).

```python

interface_speeds = [100, 1000, 100, 10000]  # Mbps
total_bandwidth = sum(interface_speeds)
print(f"Total Bandwidth Usage: {total_bandwidth} Mbps")
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
Total Bandwidth Usage: 11200 Mbps
devasc@labvm:~/Downloads/python_class$
```

2. Intermediate: Implement a function to convert IP address octet values to decimal.

```python
#2. Intermediate:
# Implement a function to convert IP address octet values to decimal.
def ip_to_decimal(ip):
    octets = ip.split(".")
    return sum(int(octet) << (8 * (3 - i)) for i, octet in enumerate(octets))

print(ip_to_decimal("192.168.1.1"))  # 3232235777
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
3232235777
```

3. Advanced: Calculate the subnet mask given a CIDR notation (e.g., /24).

```python
def cidr_to_mask(cidr):
    mask = (0xffffffff >> (32 - cidr)) << (32 - cidr)
    return ".".join(str((mask >> (8 * i)) & 0xff) for i in range(3, -1, -1))

print(cidr_to_mask(29))  # 255.255.255.0
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
255.255.255.248
```

String Data Type

4. Beginner:Parse a Cisco device's hostname from a given configuration string.

```
class_1.py > ...
config = """
hostname Branch-Router
interface GigabitEthernet0/0
 ip address 192.168.1.1 255.255.255.0
"""

for line in config.splitlines():
    if line.startswith("hostname"):
        hostname = line.split()[1]
        print("Hostname:", hostname)
```

```
devasc@labvm:~/Downloads/python_class$ /bin/pyth
ass_1.py
Hostname: Branch-Router
```

5. Intermediate: Extract interface names from a Cisco 'show ip int brief' output.

```
output = """
Gig0/0       192.168.1.1    YES manual up up
Gig0/1       unassigned     YES unset  administratively down down
Loopback0    1.1.1.1        YES manual up up
"""

interfaces = []

for line in output.splitlines():
    if line:
        interfaces.append(line.split()[0])

print(interfaces)
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
['Gig0/0', 'Gig0/1', 'Loopback0']
devasc@labvm:~/Downloads/python_class$
```

6. Advanced: Implement a function to validate a Cisco IOS configuration string.

```python
def validate_config(config):
    required_keywords = ["hostname", "interface", "ip address"]

    for keyword in required_keywords:
        if keyword not in config:
            return False
    return True

config = {"hostname":"R1", "interface": "Gig 0/0", "ip address": "192.168.1.1"}
print(validate_config(config))
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
True
devasc@labvm:~/Downloads/python_class$
```

List

7. Beginner: Given a list of IP addresses, filter out the private IP addresses.

```python
import ipaddress

ips = ["192.168.1.1", "8.8.8.8", "10.0.0.5", "172.168.0.1", "197.210.211.1", "192.168.2.2"]
private_ips = [ip for ip in ips if ipaddress.ip_address(ip).is_private]
print(private_ips)  # ['192.168.1.1', '10.0.0.5']
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
['192.168.1.1', '10.0.0.5', '192.168.2.2']
devasc@labvm:~/Downloads/python_class$
```

8. Intermediate: Implement a function to generate a list of IP addresses from a given subnet.

```python
import ipaddress

def generate_ips(subnet):
    return [str(ip) for ip in ipaddress.ip_network(subnet).hosts()]

print(generate_ips("192.168.1.0/28"))
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
['192.168.1.1', '192.168.1.2', '192.168.1.3', '192.168.1.4', '192.168.1.5', '192.168.1.6', '192.168.1.7', '192.168.1.8
', '192.168.1.9', '192.168.1.10', '192.168.1.11', '192.168.1.12', '192.168.1.13', '192.168.1.14']
devasc@labvm:~/Downloads/python_class$
```

9. Advanced: Find duplicate IP addresses in a list of device configurations.

```python
ips = ["192.168.1.1", "10.0.0.1", "192.168.1.1", "8.8.8.8", "2.2.2.2", "10.0.0.1"]
duplicates = set([ip for ip in ips if ips.count(ip) > 1])
print(duplicates)
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
{'192.168.1.1', '10.0.0.1'}
```

Dictionary

10. Beginner: Create a dictionary mapping interface names to their IP addresses from a Cisco device configuration.

```python
config = """
interface Gig0/0
 ip address 192.168.1.1 255.255.255.0
interface Gig0/1
 ip address 10.0.0.1 255.255.255.0
"""

interfaces = {}
current_int = None

for line in config.splitlines():
    if line.startswith("interface"):
        current_int = line.split()[1]
    elif "ip address" in line:
        ip = line.split()[2]
        interfaces[current_int] = ip

print(interfaces)
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
{'Gig0/0': '192.168.1.1', 'Gig0/1': '10.0.0.1'}
```

11. Intermediate: Merge two dictionaries containing device configurations.

```python
device1 = {"Gig0/0": "192.168.1.1"}
device2 = {"Gig0/1": "10.0.0.1"}

merged = {**device1, **device2}

print(merged)
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
{'Gig0/0': '192.168.1.1', 'Gig0/1': '10.0.0.1'}
```

12. Advanced: Implement a function to compare two device configurations and return the differences.

```python
def compare_configs(cfg1, cfg2):
    differences = {}
    for key in set(cfg1.keys()).union(cfg2.keys()):
        if cfg1.get(key) != cfg2.get(key):
            differences[key] = (cfg1.get(key), cfg2.get(key))
    return differences

cfg1 = {"hostname": "R1", "ip": "192.168.1.1"}
cfg2 = {"hostname": "R2", "ip": "192.168.1.1"}
print(compare_configs(cfg1, cfg2))
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
{'hostname': ('R1', 'R2')}
```

If Conditional

13. Beginner: Check if a given IP address is valid.

```python
import ipaddress

def is_valid_ip(ip):
    try:
        ipaddress.ip_address(ip)
        return True
    except ValueError:
        return False

print(is_valid_ip("192.168.1.256"))
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
False
```

14. Intermediate: Determine if a Cisco device is reachable based on ping output.

```python
ping_output = "Reply from 192.168.1.1: bytes=32 time<1ms TTL=64"
if "Reply from" in ping_output:
    print("Device is reachable")
else:
    print("Device is not reachable")
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/Downloads/python_class/class_1.py
Device is reachable
```

15. Advanced: Implement a function to check if a given ACL (Access Control List) rule is valid for a specific interface.

```python
def validate_acl_rule(rule, interface_ip):
    if "deny" in rule:
        return True  # Deny rules are generally safe
    elif interface_ip in rule:
        return True
    else:
        return False

rule = "permit ip 192.168.1.0 0.0.0.255 any"
print(validate_acl_rule(rule, "192.168.1.1"))
```

```
devasc@labvm:~/Downloads/python_class$ /bin/python3 /home/devasc/
False
```